```
import java.io.BufferedReader;
import java.io.InputStreamReader;
import java.io.IOException;

public class Main {

    static BufferedReader in = new BufferedReader(
            new InputStreamReader(System.in));

    // Shared document store used by TextHack.
    static final int MAX_DOCS = 100;
    static String[] documents = new String[MAX_DOCS];
    static int docCount = 0;

    static String read(String prompt) throws IOException {
        System.out.print(prompt);
        String s = in.readLine();
        return s == null ? "" : s;
    }

    static int number(String prompt) throws IOException {
        return Integer.parseInt(read(prompt).trim());
    }

    static String multiline(String prompt) throws IOException {
        System.out.println(prompt);
        System.out.println("Type END on a separate line to finish.");

        StringBuilder b = new StringBuilder();
        String line;

        while ((line = in.readLine()) != null) {
            if (line.equals("END")) break;
            if (b.length() > 0) b.append('\n');
            b.append(line);
        }

        return b.toString();
    }

    static int[] points(String s) {
        return s.codePoints().toArray();
    }
```

```
static void addDocument(String text) {
    if (docCount == MAX_DOCS) {
        System.out.println("Document store is full.");
        return;
    }

    documents[docCount] = text;
    System.out.println("Document added with ID: " + docCount);
    docCount++;
}

static void showDocuments() {
    if (docCount == 0) {
        System.out.println("No documents stored.");
        return;
    }

    for (int i = 0; i < docCount; i++) {
        System.out.println("\nDocument " + i + ":");
        System.out.println(documents[i]);
    }
}

// ====================================================
// MODULE 2: AHO-CORASICK MULTI-PATTERN SEARCH
// Custom trie, failure links, output lists and BFS queue.
// ====================================================

static class Edge {
    int cp;
    Node target;
    Edge next;

    Edge(int cp, Node target, Edge next) {
        this.cp = cp;
        this.target = target;
        this.next = next;
    }
}

static class Output {
    int id;
    Output next;
```

```
    Output(int id, Output next) {
        this.id = id;
        this.next = next;
    }
}

static class Node {
    Edge edges;
    Node failure;
    Output outputs;

    Node get(int cp) {
        for (Edge e = edges; e != null; e = e.next)
            if (e.cp == cp) return e.target;
        return null;
    }

    void addEdge(int cp, Node target) {
        edges = new Edge(cp, target, edges);
    }

    void addOutput(int id) {
        outputs = new Output(id, outputs);
    }
}

static class AhoCorasick {
    Node root = new Node();
    Node[] queue;
    String[] patterns;
    int[][] patternPoints;
    int nodes = 1;

    AhoCorasick(String[] patterns) {
        this.patterns = patterns;
        patternPoints = new int[patterns.length][];

        for (int i = 0; i < patterns.length; i++) {
            if (patterns[i] == null || patterns[i].isEmpty())
                throw new IllegalArgumentException(
                        "Patterns cannot be empty.");

            patternPoints[i] = points(patterns[i]);
        }
```

```
    buildTrie();
    buildFailures();
}

void buildTrie() {
    for (int id = 0; id < patterns.length; id++) {
        Node current = root;

        for (int cp : patternPoints[id]) {
            Node next = current.get(cp);

            if (next == null) {
                next = new Node();
                current.addEdge(cp, next);
                nodes++;
            }

            current = next;
        }

        current.addOutput(id);
    }
}

void buildFailures() {
    queue = new Node[nodes];
    int head = 0, tail = 0;

    for (Edge e = root.edges; e != null; e = e.next) {
        e.target.failure = root;
        queue[tail++] = e.target;
    }

    while (head < tail) {
        Node current = queue[head++];

        for (Edge e = current.edges; e != null; e = e.next) {
            int cp = e.cp;
            Node child = e.target;
            Node fallback = current.failure;

            while (fallback != root && fallback.get(cp) == null)
                fallback = fallback.failure;
```

```
                Node target = fallback.get(cp);
                child.failure = target == null ? root : target;

                for (Output o = child.failure.outputs;
                     o != null; o = o.next)
                    child.addOutput(o.id);

                queue[tail++] = child;
            }
        }
    }

    int search(String text, int docId) {
        int[] t = points(text);
        Node current = root;
        int count = 0;

        for (int i = 0; i < t.length; i++) {
            int cp = t[i];

            while (current != root && current.get(cp) == null)
                current = current.failure;

            Node next = current.get(cp);
            current = next == null ? root : next;

            for (Output o = current.outputs;
                 o != null; o = o.next) {
                int start = i - patternPoints[o.id].length + 1;

                System.out.println(
                        "Document " + docId +
                        " | Pattern: " + patterns[o.id] +
                        " | Start: " + start +
                        " | End: " + i);

                count++;
            }
        }

        return count;
    }
}
```

```
static void multiPatternSearch() throws IOException {
    if (docCount == 0) {
        System.out.println("Add documents first.");
        return;
    }

    int n = number("Number of patterns: ");
    if (n < 1 || n > 500) {
        System.out.println("Enter between 1 and 500 patterns.");
        return;
    }

    String[] patterns = new String[n];

    for (int i = 0; i < n; i++)
        patterns[i] = read("Pattern " + (i + 1) + ": ");

    AhoCorasick engine = new AhoCorasick(patterns);
    int total = 0;

    for (int d = 0; d < docCount; d++)
        total += engine.search(documents[d], d);

    System.out.println("Total matches: " + total);
    System.out.println("Algorithm: Aho-Corasick");
    System.out.println("Search complexity: O(N + Z), after preprocessing");
}

// ====================================================
// MODULE 2: KMP SINGLE-PATTERN SEARCH
// ====================================================

static int kmp(String text, String pattern) {
    int[] t = points(text);
    int[] p = points(pattern);

    if (p.length == 0) {
        System.out.println("Pattern cannot be empty.");
        return 0;
    }

    int[] lps = new int[p.length];
```

```
    for (int i = 1, len = 0; i < p.length;) {
        if (p[i] == p[len]) {
            lps[i++] = ++len;
        } else if (len > 0) {
            len = lps[len - 1];
        } else {
            lps[i++] = 0;
        }
    }

    int i = 0, j = 0, count = 0;

    while (i < t.length) {
        if (t[i] == p[j]) {
            i++;
            j++;
        }

        if (j == p.length) {
            System.out.println("Match at code-point index " + (i - j));
            count++;
            j = lps[j - 1];
        } else if (i < t.length && t[i] != p[j]) {
            if (j > 0) j = lps[j - 1];
            else i++;
        }
    }

    System.out.println("Total matches: " + count);
    System.out.println("Time complexity: O(n + m)");
    return count;
}

static void kmpQuery() throws IOException {
    String text = multiline("Enter your search text.");
    String pattern = read("Enter the pattern: ");
    kmp(text, pattern);
}

// ======================================================
// MODULE 3: EDIT DISTANCE / FUZZY MATCHING
// ======================================================

static int editDistance(String a, String b) {
```

```
    int[] x = points(a);
    int[] y = points(b);
    int[][] dp = new int[x.length + 1][y.length + 1];

    for (int i = 0; i <= x.length; i++) dp[i][0] = i;
    for (int j = 0; j <= y.length; j++) dp[0][j] = j;

    for (int i = 1; i <= x.length; i++) {
        for (int j = 1; j <= y.length; j++) {
            int cost = x[i - 1] == y[j - 1] ? 0 : 1;

            dp[i][j] = Math.min(
                    dp[i - 1][j] + 1,
                    Math.min(dp[i][j - 1] + 1,
                            dp[i - 1][j - 1] + cost));
        }
    }

    return dp[x.length][y.length];
}

static void fuzzySearch() throws IOException {
    if (docCount == 0) {
        System.out.println("Add documents first.");
        return;
    }

    String query = read("Enter your search word: ");
    int limit = number("Maximum edit distance allowed: ");

    if (limit < 0) {
        System.out.println("Distance must be non-negative.");
        return;
    }

    boolean found = false;

    for (int d = 0; d < docCount; d++) {
        String[] words = documents[d].split("\\s+");

        for (String word : words) {
            String candidate = word.replaceAll(
                    "^[\\p{Punct}]+|[\\p{Punct}]+$", "");
```

```
            if (candidate.isEmpty()) continue;

            int distance = editDistance(
                    query.toLowerCase(),
                    candidate.toLowerCase());

            if (distance <= limit) {
                System.out.println(
                        "Document " + d + ": " + candidate +
                        " | Edit distance: " + distance);
                found = true;
            }
        }
    }

    if (!found) System.out.println("No similar words found.");
    System.out.println("Algorithm: Wagner-Fischer DP");
    System.out.println("Per comparison: O(n * m)");
}

// ========================================================
// MODULE 3: DOCUMENT SIMILARITY USING LCS
// ========================================================

static void similarity() throws IOException {
    if (docCount < 2) {
        System.out.println("Add at least two documents.");
        return;
    }

    int a = number("First document ID (0 to " + (docCount - 1) + "): ");
    int b = number("Second document ID (0 to " + (docCount - 1) + "): ");

    if (a < 0 || a >= docCount || b < 0 || b >= docCount) {
        System.out.println("Invalid document ID.");
        return;
    }

    int[] x = points(documents[a]);
    int[] y = points(documents[b]);
    int[][] dp = new int[x.length + 1][y.length + 1];

    for (int i = 1; i <= x.length; i++) {
        for (int j = 1; j <= y.length; j++) {
```

```
            if (x[i - 1] == y[j - 1])
                dp[i][j] = dp[i - 1][j - 1] + 1;
            else
                dp[i][j] = Math.max(dp[i - 1][j], dp[i][j - 1]);
        }
    }

    int lcs = dp[x.length][y.length];
    int sum = x.length + y.length;
    double score = sum == 0 ? 100 : 200.0 * lcs / sum;

    System.out.println("Longest common subsequence: " + lcs);
    System.out.printf("LCS similarity: %.2f%%%n", score);
    System.out.println("Time complexity: O(n * m)");
}

// ====================================================
// MODULE 4: EDMONDS-KARP MAXIMUM FLOW
// ====================================================

static boolean bfs(int[][] r, int s, int t, int[] parent, int n) {
    boolean[] seen = new boolean[n];
    int[] q = new int[n];
    int head = 0, tail = 0;

    q[tail++] = s;
    seen[s] = true;
    parent[s] = -1;

    while (head < tail) {
        int u = q[head++];

        for (int v = 0; v < n; v++) {
            if (!seen[v] && r[u][v] > 0) {
                seen[v] = true;
                parent[v] = u;

                if (v == t) return true;
                q[tail++] = v;
            }
        }
    }

    return false;
```

```
}

static void maxFlowDemo() {
    int n = 6;
    int[][] c = new int[n][n];

    c[0][1] = 16;
    c[0][2] = 13;
    c[1][2] = 10;
    c[2][1] = 4;
    c[1][3] = 12;
    c[3][2] = 9;
    c[2][4] = 14;
    c[4][3] = 7;
    c[3][5] = 20;
    c[4][5] = 4;

    int[][] r = new int[n][n];

    for (int i = 0; i < n; i++)
        for (int j = 0; j < n; j++)
            r[i][j] = c[i][j];

    int[] parent = new int[n];
    int flow = 0;

    while (bfs(r, 0, 5, parent, n)) {
        int amount = Integer.MAX_VALUE;

        for (int v = 5; v != 0; v = parent[v])
            amount = Math.min(amount, r[parent[v]][v]);

        for (int v = 5; v != 0; v = parent[v]) {
            int u = parent[v];
            r[u][v] -= amount;
            r[v][u] += amount;
        }

        flow += amount;
    }

    System.out.println("Maximum flow: " + flow);
    System.out.println("Algorithm: Edmonds-Karp");
    System.out.println("Complexity: O(V * E^2)");
```

```
}

// ====================================================
// MODULE 4: BIPARTITE MATCHING
// ====================================================

static boolean augment(int u, boolean[][] graph,
                       boolean[] seen, int[] match) {
    for (int v = 0; v < match.length; v++) {
        if (graph[u][v] && !seen[v]) {
            seen[v] = true;

            if (match[v] == -1 ||
                augment(match[v], graph, seen, match)) {
                match[v] = u;
                return true;
            }
        }
    }

    return false;
}

static void matchingDemo() {
    boolean[][] graph = {
            {true, true, false},
            {true, false, true},
            {false, true, true}
    };

    int[] match = {-1, -1, -1};
    int count = 0;

    for (int u = 0; u < graph.length; u++) {
        boolean[] seen = new boolean[match.length];
        if (augment(u, graph, seen, match)) count++;
    }

    System.out.println("Maximum matching size: " + count);

    for (int v = 0; v < match.length; v++)
        if (match[v] != -1)
            System.out.println("Left " + match[v] + " -> Right " + v);
}
```

```
// ========================================================
// MODULE 5: VERTEX-COVER 2-APPROXIMATION
// ========================================================

static void vertexCoverDemo() {
    int[] u = {0, 0, 1, 1, 2, 3, 3};
    int[] v = {1, 2, 3, 4, 4, 5, 6};

    boolean[] selected = new boolean[7];

    for (int i = 0; i < u.length; i++) {
        if (!selected[u[i]] && !selected[v[i]]) {
            selected[u[i]] = true;
            selected[v[i]] = true;
        }
    }

    System.out.print("Approximate vertex cover: ");

    for (int i = 0; i < selected.length; i++)
        if (selected[i]) System.out.print(i + " ");

    System.out.println();
    System.out.println("Guarantee: at most 2 times the optimal size.");
}

// ========================================================
// MODULE 5: DEADLINE-ORDER SCHEDULING DEMONSTRATION
// ========================================================

static void schedulingDemo() {
    String[] jobs = {"Corpus", "Index", "Search", "Testing", "Report"};
    int[] duration = {3, 2, 4, 2, 1};
    int[] deadline = {5, 4, 10, 8, 12};

    for (int i = 0; i < jobs.length - 1; i++) {
        for (int j = i + 1; j < jobs.length; j++) {
            if (deadline[j] < deadline[i]) {
                int t = deadline[i];
                deadline[i] = deadline[j];
                deadline[j] = t;

                t = duration[i];
```

```
                duration[i] = duration[j];
                duration[j] = t;

                String s = jobs[i];
                jobs[i] = jobs[j];
                jobs[j] = s;
            }
        }
    }

    int elapsed = 0;

    for (int i = 0; i < jobs.length; i++) {
        elapsed += duration[i];

        System.out.println(jobs[i] + ": finish=" + elapsed
                + ", deadline=" + deadline[i]
                + (elapsed > deadline[i] ? " (late)" : " (on time)"));
    }

    System.out.println("This is a heuristic demonstration.");
}

// ========================================================
// MODULE 6: MILLER-RABIN PRIMALITY TEST
// ========================================================

static long addMod(long a, long b, long mod) {
    return a >= mod - b ? a - (mod - b) : a + b;
}

static long multiplyMod(long a, long b, long mod) {
    long result = 0;
    a %= mod;

    while (b > 0) {
        if ((b & 1L) != 0) result = addMod(result, a, mod);
        b >>= 1;

        if (b > 0) a = addMod(a, a, mod);
    }

    return result;
}
```

```
static long powerMod(long base, long exp, long mod) {
    long result = 1;
    base %= mod;

    while (exp > 0) {
        if ((exp & 1L) != 0)
            result = multiplyMod(result, base, mod);

        exp >>= 1;

        if (exp > 0) base = multiplyMod(base, base, mod);
    }

    return result;
}

static boolean millerRabin(long n, int rounds) {
    if (n < 2) return false;
    if (n == 2 || n == 3) return true;
    if (n % 2 == 0) return false;

    long d = n - 1;
    int s = 0;

    while (d % 2 == 0) {
        d /= 2;
        s++;
    }

    for (int r = 0; r < rounds; r++) {
        long a = 2 + (long) (Math.random() * (n - 3));
        long x = powerMod(a, d, n);

        if (x == 1 || x == n - 1) continue;

        boolean passed = false;

        for (int j = 1; j < s; j++) {
            x = multiplyMod(x, x, n);

            if (x == n - 1) {
                passed = true;
                break;
```

```
            }
        }

        if (!passed) return false;
    }

    return true;
}

// ========================================================
// MODULE 6: BLELLOCH PREFIX SCAN
// ========================================================

static void prefixScan() throws IOException {
    int n = number("Number of elements: ");

    if (n < 1 || n > 100000) {
        System.out.println("Enter 1 to 100000 elements.");
        return;
    }

    long[] a = new long[n];

    for (int i = 0; i < n; i++)
        a[i] = Long.parseLong(read("Element " + i + ": ").trim());

    int size = 1;
    while (size < n) size *= 2;

    long[] tree = new long[size];

    for (int i = 0; i < n; i++) tree[i] = a[i];

    // Up-sweep.
    for (int step = 1; step < size; step *= 2)
        for (int i = 0; i < size; i += 2 * step)
            tree[i + 2 * step - 1] += tree[i + step - 1];

    tree[size - 1] = 0;

    // Down-sweep.
    for (int step = size / 2; step >= 1; step /= 2) {
        for (int i = 0; i < size; i += 2 * step) {
            long temp = tree[i + step - 1];
```

```
            tree[i + step - 1] = tree[i + 2 * step - 1];
            tree[i + 2 * step - 1] += temp;
        }
    }

    System.out.print("Prefix sums: ");

    for (int i = 0; i < n; i++)
        System.out.print((tree[i] + a[i]) + " ");

    System.out.println();
    System.out.println("Work complexity: O(n)");
    System.out.println("Sequential demonstration of a parallel algorithm.");
}

// ======================================================
// MODULE 1: PUBLIC API
// ======================================================

static void apiOverview() {
    System.out.println("\n=== TEXTHACK PUBLIC API ===");
    System.out.println("Document management: add and list documents");
    System.out.println("Pattern search: Aho-Corasick and KMP");
    System.out.println("Fuzzy search: edit distance DP");
    System.out.println("Document similarity: LCS DP");
    System.out.println("Network optimization: max-flow and matching");
    System.out.println("Approximation: vertex cover");
    System.out.println("Scheduling: deadline-order heuristic");
    System.out.println("Randomized algorithms: Miller-Rabin");
    System.out.println("Parallel algorithms: prefix-scan demonstration");
}

static void menu() {
    System.out.println("\n========== TEXTHACK ==========");
    System.out.println("0. API overview");
    System.out.println("1. Add your own document");
    System.out.println("2. View stored documents");
    System.out.println("3. Unicode multi-pattern search");
    System.out.println("4. KMP single-pattern search");
    System.out.println("5. Fuzzy search across documents");
    System.out.println("6. Compare two documents");
    System.out.println("7. Maximum flow");
    System.out.println("8. Bipartite matching");
    System.out.println("9. Vertex-cover approximation");
```

```
    System.out.println("10. Scheduling demonstration");
    System.out.println("11. Miller-Rabin primality test");
    System.out.println("12. Prefix scan");
    System.out.println("13. Exit");
}

public static void main(String[] args) throws IOException {
    while (true) {
        menu();

        int choice;

        try {
            choice = number("Choose an option: ");
        } catch (NumberFormatException e) {
            System.out.println("Enter a valid menu number.");
            continue;
        }

        try {
            switch (choice) {
                case 0:
                    apiOverview();
                    break;

                case 1: {
                    if (docCount >= MAX_DOCS) {
                        System.out.println("Document store is full.");
                        break;
                    }

                    String text = multiline("Enter your document.");
                    if (text.isEmpty()) {
                        System.out.println("Empty document not added.");
                    } else {
                        addDocument(text);
                    }
                    break;
                }

                case 2:
                    showDocuments();
                    break;
```

```
case 3:
   multiPatternSearch();
   break;

case 4:
   kmpQuery();
   break;

case 5:
   fuzzySearch();
   break;

case 6:
   similarity();
   break;

case 7:
   maxFlowDemo();
   break;

case 8:
   matchingDemo();
   break;

case 9:
   vertexCoverDemo();
   break;

case 10:
   schedulingDemo();
   break;

case 11: {
   long n = Long.parseLong(
         read("Enter a positive integer: ").trim());

   if (n < 2) {
      System.out.println("Not prime.");
   } else if (n == 2 || n == 3) {
      System.out.println("Prime.");
   } else {
      System.out.println(
            millerRabin(n, 10)
                  ? "Probably prime (10 rounds)."
```

```
                                : "Composite.");
                    }

                    break;
                }

                case 12:
                    prefixScan();
                    break;

                case 13:
                    System.out.println("TextHack terminated.");
                    return;

                default:
                    System.out.println("Invalid option.");
            }
        } catch (NumberFormatException e) {
            System.out.println("Invalid number.");
        } catch (IllegalArgumentException e) {
            System.out.println("Input error: " + e.getMessage());
        }
    }
  }
}
```